

Team: Lasiru Weerasuriya, Jack Gately

CSC 665 Assignment 4

Problem I, Question 1

The hinge loss for binary classification is

$$\ell(h(x), y) = \max(0, 1 - yh(x)),$$

where the label is

$$y \in \{-1, +1\}.$$

The value

$$yh(x)$$

is the margin. If the margin is at least 1, then the point is not only classified correctly, but it is also far enough from the decision boundary, so the hinge loss is 0. If the margin is between 0 and 1, the point is technically classified correctly, but it is still too close to the boundary, so it gets penalized. If the margin is less than or equal to 0, then the point is misclassified and also gets penalized.

This makes hinge loss a good objective for large-margin binary classification because it cares about more than just getting the sign right. The usual 0-1 classification loss only checks whether the prediction is correct or incorrect, but it is hard to optimize directly. Hinge loss is a convex surrogate for that loss, which makes the optimization problem much easier to work with while still pushing the classifier toward a larger margin. This is the main idea behind support vector machines, where the learned classifier tries to separate the two classes while keeping the boundary as far as possible from the training points. Cortes and Vapnik introduced support-vector networks for this kind of two-class classification problem [1]. Shalev-Shwartz and Ben-David also explain why convex surrogate losses are useful when the original classification objective is difficult to optimize directly [2]. Other SVM references describe how the hinge-loss setup leads to a tractable convex optimization problem [3]. Hinge loss is also convenient for gradient-based and online learning methods because its subgradient is simple to compute [4].

Problem I, Question 2

The model is

$$h(x_1, x_2) = w_0 + w_1x_1 + w_2x_2.$$

The hinge loss is

$$c(h(x_1, x_2), y) = \max(0, 1 - yh(x_1, x_2)).$$

Let the margin be

$$m = yh(x_1, x_2).$$

Then the hinge loss can be written in two cases:

$$c(h(x_1, x_2), y) = \begin{cases} 0, & \text{if } yh(x_1, x_2) \geq 1, \\ 1 - yh(x_1, x_2), & \text{if } yh(x_1, x_2) < 1. \end{cases}$$

When

$$y h(x_1, x_2) > 1,$$

the loss is flat, so all three partial derivatives are 0:

$$\begin{aligned} \frac{\partial c}{\partial w_0} &= 0, \\ \frac{\partial c}{\partial w_1} &= 0, \\ \frac{\partial c}{\partial w_2} &= 0. \end{aligned}$$

When

$$y h(x_1, x_2) < 1,$$

the loss is

$$1 - y(w_0 + w_1 x_1 + w_2 x_2).$$

So the partial derivatives are:

$$\begin{aligned} \frac{\partial c}{\partial w_0} &= -y, \\ \frac{\partial c}{\partial w_1} &= -y x_1, \\ \frac{\partial c}{\partial w_2} &= -y x_2. \end{aligned}$$

At the exact point where

$$y h(x_1, x_2) = 1,$$

the hinge loss has a corner, so the regular derivative does not exist. We use a subgradient there. A valid subgradient can be any value between the zero side and the nonzero side. Using

$$x_0 = 1,$$

we can write this as

$$\begin{aligned} \frac{\partial c}{\partial w_j} &\in \alpha(-y x_j), \\ \alpha &\in [0, 1]. \end{aligned}$$

For example, at the boundary it is valid to choose the zero subgradient.

Problem I, Question 3

Using the hinge-loss subgradient with learning rate $\eta = 0.1$:

If $y * h(x_1, x_2) < 1$, update

$$w \leftarrow w + \eta * y * (1, x_1, x_2)$$

If $y * h(x_1, x_2) \geq 1$, the update is zero.

Weights after each SGD update:

iteration	example	w0	w1	w2
1	(-4, 0, -1)	-0.1	0.4	0.0
2	(-1, 1, +1)	0.0	0.3	0.1
3	(0, -1, -1)	-0.1	0.3	0.2
4	(2, 1, +1)	0.0	0.5	0.3

iteration	example	w0	w1	w2
5	(3, 0, +1)	0.0	0.5	0.3
6	(6, -1, -1)	-0.1	-0.1	0.4

Final SGD weights:

$(w_0, w_1, w_2) = (-0.1, -0.1, 0.4)$

Problem I, Question 4

Using $h(x_1, x_2) = -0.1 - 0.1x_1 + 0.4x_2$ and predicting +1 when $h \geq 0$:

i	$h(x_1, x_2)$	prediction	y	correct?
1	0.3	+1	-1	no
2	0.4	+1	+1	yes
3	-0.5	-1	-1	yes
4	0.1	+1	+1	yes
5	-0.4	-1	+1	no
6	-1.1	-1	-1	yes

There are 2 mistakes out of 6 examples.

misclassification rate = $2/6 = 1/3 = 0.333333$

Problem I, Question 5 EC

The file `problem1_gd.py` runs batch gradient descent using the full dataset on each update.

With $\eta = 0.1$ and stopping threshold $1e-5$:

SGD final weights: $(-0.1, -0.1, 0.4)$

SGD training error: 0.333333

SGD hinge loss: 4.700000

Batch GD final weights: $(-0.4, 0.5, 4.0)$

Batch GD updates: 58

Batch GD training error: 0.000000

Batch GD hinge loss: 0.000000

Problem II, Question 1

The cost function is

$$C(w) = \sum_{i=1}^n \left(y_i - (w_0 + w_1 x_i) \right)^2.$$

Since this problem has

$$n = 2,$$

we can write it as

$$\begin{aligned} C(w) = & \\ & \left(y_1 - (w_0 + w_1 x_1) \right)^2 \\ & + \\ & \left(y_2 - (w_0 + w_1 x_2) \right)^2. \end{aligned}$$

First, take the derivative with respect to

$$w_0.$$

For each term, the inside expression has derivative

$$-1$$

with respect to

$$w_0.$$

So

$$\begin{aligned} \frac{dC}{dw_0} &= \\ & -2 \left(y_1 - (w_0 + w_1 x_1) \right) \\ & -2 \left(y_2 - (w_0 + w_1 x_2) \right). \end{aligned}$$

An equivalent form is

$$\begin{aligned} \frac{dC}{dw_0} &= \\ & 2(w_0 + w_1 x_1 - y_1) \\ & + \\ & 2(w_0 + w_1 x_2 - y_2). \end{aligned}$$

Now take the derivative with respect to

$$w_1.$$

For each term, the inside expression has derivative

$$-x_i$$

with respect to

$$w_1.$$

Therefore,

$$\begin{aligned} \frac{dC}{dw_1} &= \\ & -2x_1 \left(y_1 - (w_0 + w_1 x_1) \right) \\ & -2x_2 \left(y_2 - (w_0 + w_1 x_2) \right). \end{aligned}$$

An equivalent form is

$$\begin{aligned} \frac{dC}{dw_1} &= \end{aligned}$$

$$2x_1(w_0 + w_1x_1 - y_1) + 2x_2(w_0 + w_1x_2 - y_2).$$

Problem II, Question 2

The regularized cost function is

$$\begin{aligned} \tilde{C}(w) &= \sum_{i=1}^n \left(y_i - (w_0 + w_1x_i) \right)^2 \\ &+ \lambda(w_0^2 + w_1^2). \end{aligned}$$

Since

$$n = 2,$$

this is

$$\begin{aligned} \tilde{C}(w) &= \left(y_1 - (w_0 + w_1x_1) \right)^2 \\ &+ \left(y_2 - (w_0 + w_1x_2) \right)^2 \\ &+ \lambda(w_0^2 + w_1^2). \end{aligned}$$

The regularization term adds

$$2\lambda w_0$$

to the derivative with respect to

$$w_0$$

and

$$2\lambda w_1$$

to the derivative with respect to

$$w_1.$$

So the derivative with respect to

$$w_0$$

is

$$\begin{aligned} \frac{d\tilde{C}}{dw_0} &= -2\left(y_1 - (w_0 + w_1x_1) \right) \\ &- 2\left(y_2 - (w_0 + w_1x_2) \right) \\ &+ 2\lambda w_0. \end{aligned}$$

Equivalently,

$$\begin{aligned} \frac{d\tilde{C}}{dw_0} &= 2(w_0 + w_1x_1 - y_1) \\ &+ 2(w_0 + w_1x_2 - y_2) \\ &+ 2\lambda w_0. \end{aligned}$$

The derivative with respect to

w_1

is

$$\begin{aligned} & \frac{d\tilde{C}}{dw_1} \\ &= \\ & -2x_1(y_1 - (w_0 + w_1x_1)) \\ & -2x_2(y_2 - (w_0 + w_1x_2)) \\ & + \\ & 2\lambda w_1. \end{aligned}$$

Equivalently,

$$\begin{aligned} & \frac{d\tilde{C}}{dw_1} \\ &= \\ & 2x_1(w_0 + w_1x_1 - y_1) \\ & + \\ & 2x_2(w_0 + w_1x_2 - y_2) \\ & + \\ & 2\lambda w_1. \end{aligned}$$

In summation form, the same answer is

$$\begin{aligned} & \frac{d\tilde{C}}{dw_0} \\ &= \\ & -2\sum_{i=1}^2(y_i - (w_0 + w_1x_i)) \\ & + \\ & 2\lambda w_0, \end{aligned}$$

and

$$\begin{aligned} & \frac{d\tilde{C}}{dw_1} \\ &= \\ & -2\sum_{i=1}^2x_i(y_i - (w_0 + w_1x_i)) \\ & + \\ & 2\lambda w_1. \end{aligned}$$

Problem II, Question 3

For two points (x_1, y_1) and (x_2, y_2) , the unregularized best-fit line that passes through both points is:

$$\begin{aligned} w_1 &= (y_2 - y_1) / (x_2 - x_1) \\ w_0 &= y_1 - w_1x_1 \end{aligned}$$

This is implemented in `fit_without_reg`.

Problem II, Question 4

The regularized model is implemented in `fit_with_reg`. It uses 1000 gradient descent updates from $w_0 = w_1 = 0$, step size $\eta = 0.05$, and $\lambda = 1$ when called from the experiment.

Problem II, Question 5

The experiment in `regularization.py` uses 1000 trials with random seed 0 for reproducibility.

Average test errors:

```
without regularization: 1.911505
with regularization:    0.436800
```

Problem II, Question 6 EC

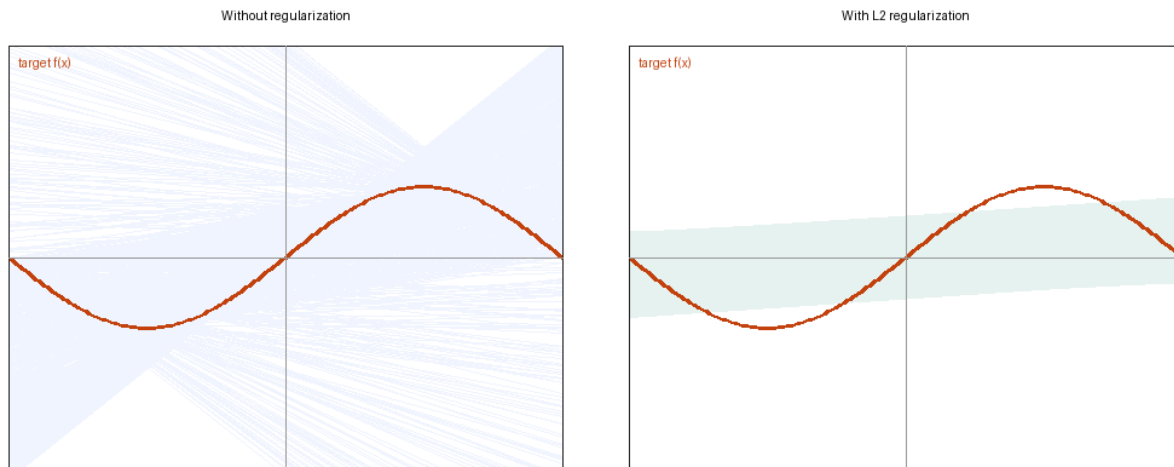
The script writes the visualization to:

```
regularization_plot.svg
```

I also generated a PNG copy for easier insertion into a document:

```
regularization_plot.png
```

The left panel shows the 1000 unregularized fitted lines. The right panel shows the 1000 L2-regularized fitted lines. In both panels, the orange curve is the target function $f(x) = \sin(\pi x)$.



References

- [1] Cortes, C., and Vapnik, V. (1995). Support-vector networks (<https://link.springer.com/article/10.1007/BF00994018>). *Machine Learning*, 20, 273-297.
- [2] Shalev-Shwartz, S., and Ben-David, S. (2014). Convex Learning Problems (<https://www.cambridge.org/core/books/understanding-machine-learning/convex-learning-problems/68712BE60A0D7640360C8305E86B3A57>), in *Understanding Machine Learning*.
- [3] Cristianini, N., and Shawe-Taylor, J. (2000). Support Vector Machines (<https://resolve.cambridge.org/core/books/abs/an-introduction-to-support-vector-machines-and-other-kernelbased-learning-methods/support-vector-machines/DD4EA48AA6C383944EA67BF8A7BEC6CC>), in *An Introduction to Support Vector Machines*.
- [4] Crammer, K., Dekel, O., Keshet, J., Shalev-Shwartz, S., and Singer, Y. (2006). Online Passive-Aggressive Algorithms (<https://www.jmlr.org/beta/papers/v7/crammer06a.html>). *Journal of Machine Learning Research*, 7, 551-585.

GenAI Usage Note

GenAI was used to help implement and verify the coding portions of Problem I Q5 and Problem II Q3-Q6, and to draft/check the numeric notes for Problem I Q3-Q4 and Problem II Q5-Q6. The code was run locally, outputs were checked against the assignment formulas, and the generated notes were reviewed for consistency with the program output.